



Value Iteration vs. Policy Iteration in Reinforcement Learning



Last updated: March 18, 2024



Written by: A. Aylin Tokuç (<https://www.baeldung.com/cs/author/aylint>)



Reviewed by: Michal Aibin (<https://www.baeldung.com/cs/editor/michal-author>)

Deep Learning (<https://www.baeldung.com/cs/category/ai/deep-learning>)

Machine Learning (<https://www.baeldung.com/cs/category/ai/ml>)

Reinforcement Learning (<https://www.baeldung.com/cs/tag/reinforcement-learning>)

1. Introduction

We can formulate a reinforcement learning (https://www.baeldung.com/cs/) (https://www.baeldung.com/cs/) problem via a Markov Decision Process (MDP). The essential elements of such a problem are the environment, state, reward, policy and value.

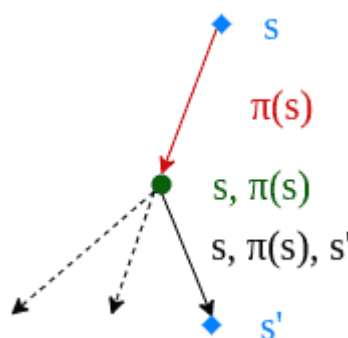
A policy (/cs/ml-policy-reinforcement-learning) is a mapping from states to actions. Finding an optimal policy leads to generating the maximum reward. **Given an MDP environment, we can use dynamic programming algorithms to compute optimal policies, which lead to the highest possible sum of future rewards at each state.**

Dynamic programming algorithms work on the assumption that we have a perfect model of the environment's MDP. So, we're able to use a one-step look-ahead approach and compute rewards for all possible actions.

In this tutorial, we'll discuss how to find an optimal policy for a given MDP. More specifically, **we'll learn about two dynamic programming algorithms: value iteration and policy iteration.** Then, we'll discuss these algorithms' advantages and disadvantages over each other.

2. Policy Iteration

In policy iteration, we start by choosing an arbitrary policy π . Then, we iteratively evaluate and improve the policy until convergence:



We evaluate a policy $\pi(s)$ by calculating the state value function $V(s)$:

$$V(s) = \sum_{s', r} p(s', r | s, \pi(s)) [r + \gamma V(s')] \quad (\text{https://www.baeldung.com/cs/})$$

Then, we calculate the improved policy by using one-step look-ahead to replace the initial policy $\pi(s)$:  (baeldung.com)

$$\pi(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

Here, r is the reward generated by taking the action a , γ is a discount factor (/cs/epsilon-greedy-q-learning#2-gamma-boldsymbolgamma) for future rewards and p is the transition probability.

In the beginning, we don't care about the initial policy π_0 being optimal or not. During the execution, we concentrate on improving it on every iteration by repeating policy evaluation and policy improvement steps. Using this algorithm we produce a chain of policies, where each policy is an improvement over the previous one:

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi_* \xrightarrow{E} v_*$$

We conduct policy evaluation and policy improvement steps until the policy doesn't improve anymore:

Data: θ : a small number**Result:** V : a value function s.t. $V \approx v_*$, π : a deterministic policys.t. $\pi \approx \pi_*$

(/bael-search)

Function *PolicyIteration* **is**

```

/* Initialization */
Initialize  $V(s)$  arbitrarily;
Randomly initialize policy  $\pi(s)$ ;
/* Policy Evaluation */
 $\Delta \leftarrow 0$ ;
while  $\Delta < \theta$  do
    for each  $s \in S$  do
         $v \leftarrow V(s)$ ;
         $V(s) \leftarrow \sum_{s',r'} p(s',r|s,\pi(s))[r + \gamma V(s')]$ ;
         $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ ;
    end
end
/* Policy Improvement */
policy-stable  $\leftarrow true$ ;
for each  $s \in S$  do
    old-action  $\leftarrow \pi(s)$ ;
     $\pi(s) \leftarrow \arg \max_a \sum_{s',r'} p(s',r|s,a)[r + \gamma V(s')]$ ;
    if old-action  $\neq \pi(s)$  then
        policy-stable  $\leftarrow false$ ;
    end
end
if policy-stable then
    return  $V \approx v_*$  and  $\pi \approx \pi_*$ ;
else
    go to Policy Evaluation;
end
end

```

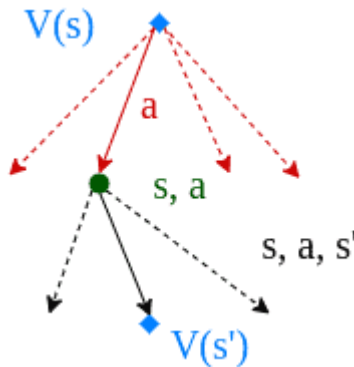
Since a finite MDP has a finite number of policies, the defined process is finite. In the end, converging an optimal policy π_* and an optimal value function v_* is guaranteed.

3. Value Iteration

(/cs/)

(https://www.baeldung.com/cs/)

In value iteration, we compute the optimal state value function by iteratively updating the estimate $v(s)$:



We start with a random value function $V(s)$. At each step, we update it:

$$V(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

Hence, we look-ahead one step and go over all possible actions at each iteration to find the maximum:

Algorithm 2: Value Iteration Algorithm

Data: θ : a small number

Result: π : a deterministic policy s.t. $\pi \approx \pi_*$

Function *ValueIteration* **is**

```
/* Initialization */
Initialize  $V(s)$  arbitrarily, except  $V(\text{terminal})$ ;
 $V(\text{terminal}) \leftarrow 0$ ;
/* Loop until convergence */
 $\Delta \leftarrow 0$ ;
while  $\Delta < \theta$  do
  for each  $s \in S$  do
     $v \leftarrow V(s)$ ;
     $V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$ ;
     $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ ;
  end
end
/* Return optimal policy */
return  $\pi$  s.t.  $\pi(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$ ;
end
```

The update step is very similar to the update step in the policy iteration algorithm. The only difference is that we take the maximum over all possible actions in the value iteration algorithm.

Instead of evaluating and then improving, the value iteration algorithm updates the state value function in a single step. This is possible by calculating all possible rewards by looking ahead.



4. Policy Iteration vs. Value Iteration

Policy iteration and value iteration are both dynamic programming algorithms that find an optimal policy π_* in a reinforcement learning environment. They both employ variations of Bellman updates and exploit one-step look-ahead:

Policy Iteration	Value Iteration
Starts with a random policy	Starts with a random value function
Algorithm is more complex	Algorithm is simpler
Guaranteed to converge	Guaranteed to converge
Cheaper to compute	More expensive to compute
Requires few iterations to converge	Requires more iterations to converge
Faster	Slower

In policy iteration, we start with a fixed policy. Conversely, in value iteration, we begin by selecting the value function. Then, in both algorithms, we iteratively improve until we reach convergence.

The policy iteration algorithm updates the policy. The value iteration algorithm iterates over the value function instead. Still, both algorithms implicitly update the policy and state value function in each iteration.

In each iteration, the policy iteration function goes through two phases. One phase evaluates the policy, and the other one improves it. The value iteration function covers these two phases by taking a maximum over the utility function for all possible actions.

The value iteration algorithm is straightforward. It combines two phases of the policy iteration into a single update operation. However, the value iteration function runs through all possible actions at once to find the maximum action value. Subsequently, the value iteration algorithm is computationally heavier.

Both algorithms are guaranteed to converge to an optimal policy in the end. Yet, the policy iteration algorithm converges within fewer iterations. As a result, the policy iteration is reported to conclude faster than the value iteration algorithm.



(/bael-search)

5. Conclusion

We use MDPs to model a reinforcement learning environment. Hence, computing the optimal policy of an MDP leads to maximizing rewards over time. We can utilize dynamic programming algorithms to finding an optimal policy.

In this article, we've investigated two algorithms to find an optimal policy for an MDP. Policy iteration and value iteration algorithms work on the same principle. We've discussed their advantages and disadvantages over each other.

Comments are closed on this article!



CATEGORIES

[ALGORITHMS \(/CS/CATEGORY/ALGORITHMS\)](#)
[ARTIFICIAL INTELLIGENCE \(/CS/CATEGORY/AI\)](#)
[CORE CONCEPTS \(/CS/CATEGORY/CORE-CONCEPTS\)](#)
[DATA STRUCTURES \(/CS/CATEGORY/DATA-STRUCTURES\)](#)
[GRAPH THEORY \(/CS/CATEGORY/GRAPH-THEORY\)](#)
[LATEX \(/CS/CATEGORY/LATEX\)](#)
[NETWORKING \(/CS/CATEGORY/NETWORKING\)](#)
[SECURITY \(/CS/CATEGORY/SECURITY\)](#)

SERIES

ABOUT

[ABOUT BAELDUNG \(HTTPS://WWW.BAELDUNG.COM/ABOUT\)](#)
[THE FULL ARCHIVE \(/CS/FULL_ARCHIVE\)](#)
[EDITORS \(HTTPS://WWW.BAELDUNG.COM/EDITORS\)](#)

[TERMS OF SERVICE \(HTTPS://WWW.BAELDUNG.COM/TERMS-OF-SERVICE\)](#)
[PRIVACY POLICY \(HTTPS://WWW.BAELDUNG.COM/PRIVACY-POLICY\)](#)
[COMPANY INFO \(HTTPS://WWW.BAELDUNG.COM/BAELDUNG-COMPANY-INFO\)](#)
[CONTACT \(/CONTACT\)](#)